

## Chapter-2

### Operating System Structures

**1. System Components:** An operating system provides the environment within which programs are executed. To construct such an environment, the system is partitioned into small modules with a well-defined interface.

A system is large and complex as an operating system can only be created by partitioning it into smaller pieces. Each of these pieces should be a well-defined portion of the system with carefully defined inputs, outputs, and function. Obviously, not all systems have the same structure. However, many modern operating systems share the system components outlined below.

- Process management
- Main Memory management
- File Management
- Storage Management
- I/O management
- Networking
- Protection
- Command Interpreter

#### 1. Process management

In general, a process will need certain resources such as CPU time, memory, files, I/O devices, etc., to accomplish its task. These resources are given to the process when it is created. In addition to the various physical and logical resources that a process obtains when it is created, some initialization data (input) may be passed along. For example, a process whose function is to display on the screen of a terminal the status of a file, say F1, will get as an input the name of the file F1 and execute the appropriate program to obtain the desired information.

The operating system is responsible for the following activities in connection with processes managed.

- The creation and deletion of both user and system processes.
- The suspension and resumption of processes.

- The provision of mechanisms for process synchronization
- The provision of mechanisms for deadlock handling.

## 2. Memory Management System

Memory is a large array of words or bytes, each with its own address. Interaction is achieved through a sequence of reads or writes of specific memory address. The CPU fetches from and stores in memory.

In order for a program to be executed it must be mapped to **absolute addresses** and loaded into memory. As the program executes, it accesses program instructions and data from memory by generating these absolute addresses.

In order to improve both the utilization of CPU and the speed of the computer's response to its users, several processes must be kept in memory. The operating system is responsible for the following activities in connection with memory management.

- Keep track of which parts of memory are currently being used and by whom.
- Decide which processes are to be loaded into memory when memory space becomes available.
- Allocate and deallocate memory space as needed.

## 3. File management

Computers can store information in several different physical forms; magnetic tape, disk, and drum are the most common forms. Each of these devices has its own characteristics and physical organization.

For convenient use of the computer system, the operating system provides a **uniform logical view** of information storage. Files are mapped, by the operating system, onto physical devices.

The operating system implements the abstract concept of the file by managing mass storage device, such as tapes and disks. Also, files are normally organized into directories to ease their use. Finally, when multiple users have access to files, it may be desirable to control by whom and in what ways files may be accessed.

The operating system is responsible for the following activities in connection with file management:

- The creation and deletion of files
- The creation and deletion of directory
- The support of primitives for manipulating files and directories
- The mapping of files onto disk storage.
- Backup of files on stable (non-volatile) storage.

#### 4. Secondary Storage Management System

The main purpose of a computer system is to execute programs. These programs, together with the data they access, must be in main memory during execution. Since the main memory is too small to permanently accommodate all data and program, the computer system must provide secondary storage to backup main memory. Most modern computer systems use **disks** as the primary **on-line storage of information**, of both programs and data. Most programs, like compilers, assemblers, sort routines, editors, formatters, and so on, are stored on the disk until loaded into memory, and then use the disk as both the source and destination of their processing. Hence the proper management of disk storage is of central importance to a computer system.

The operating system is responsible for the following activities in connection with disk management

- Free space management: How the **Operating System (OS)** keeps track of which areas of the disk are **free (unused)** and which are **occupied (used by files or directories)**.
- Storage allocation
- Disk scheduling.

#### 5. I/O System

One of the important jobs of OS is to manage various I/O devices including mouse, keyboard, touch pad etc. I/O system is required to take an application's I/O request and send it to physical device, then take whatever response come back from device and send it to application. The I/O system consists of:

- A buffer caching system
- A general device driver code
- Drivers for specific hardware devices.

A **device driver** is a **specialized software component** in the OS that acts as a **bridge between the hardware device and the operating system (or user applications)**.

It translates **general OS commands** into **specific hardware instructions** that the device can understand.

## 6. Networking

A networking system component of an OS is one that is designed for networking use. NOS is designed primarily to support workstation, PC and in some instances, older terminal that are connected to LAN. Networking system allows to share file and printer access among multiple computers in a Network, to enable the sharing of data.

## 7. Protection System

The various processes in an operating system must be protected from each other's activities. For that purpose, various mechanisms which can be used to ensure that the files, memory segment, CPU and other resources can be operated on only by those processes that have gained proper authorization from the operating system.

For example, **memory addressing hardware** ensure that a process **can only execute within its own address space**. The **timer** ensure that no process can gain control of the CPU without relinquishing it. Finally, no process is allowed to do its own I/O, to protect the integrity of the various peripheral devices.

Protection refers to a mechanism for controlling the access of programs, processes, or users to the resources defined by a computer, together with some means of enforcement.

## 8. Command Interpreter System

One of the most important components of an operating system is its command interpreter. The command interpreter is the **primary interface between the user and the rest of the system (OS)**.

Many commands are given to the operating system by control statements. When a new job is started in a batch system or when a user logs-in to a time-shared system, a **program which reads and interprets control statements is automatically executed**. This program is variously called (1) the control card interpreter, (2) the command line interpreter, (3) the shell (in Unix), and so on. Its function is quite simple: get the next command statement and execute it.

The command statement themselves deal with process management, I/O handling, secondary storage management, main memory management, file system access, protection, and networking.

## OS services

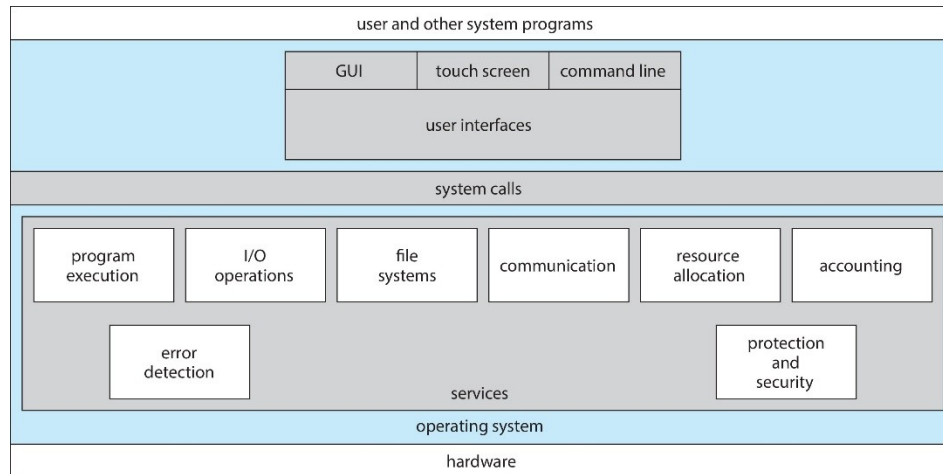
An OS provides an environment for the execution of programs. It provides certain services to programs and to the users of those programs. Following are the functions provided by OS:

1. **User interface:** almost all OS have set of user interface (UI). It can be of three types:
  - a. **Command-line interface (CLI):** which uses text commands and a method for entering them.
  - b. **Graphical user interface:** interface is a window system with a pointing device to direct I/O, choose from menu, and make selections and a keyboard to enter text.
  - c. **Touch-screen interface:** enabling users to slide their fingers across the screen or press buttons on the screen to select choices.
2. **Program execution:** The system must be able to load a program into memory and to run that program. The program must be able to end its execution, either normally or abnormally (indicating error).

3. **I/O operation:** A running program may require I/O, which may involve a file or an I/O device. For efficiency and protection, users usually cannot control I/O devices directly. Therefore, the operating system must provide some means to do I/O.
4. **File-system manipulation:** Obviously, programs need to read and write files and directories. They also need to create and delete them by name, search for a given file, and list file information. Finally, some programs include permission management to allow or deny access to files or directories based on file ownership.
5. **Communication:** In many circumstances in which one process needs to exchange information with another process. Such communication may occur between processes that are executing on same computer or between processes that are executing on different computer systems tied together by a computer network. Communication may be implemented via *shared memory* or through *message passing*, in which packets of information are moved between processes by the operating system.
6. **Error detection:** The OS needs to be constantly aware of possible errors. Errors may occur in the CPU and memory hardware (such as a memory error or a power failure), in I/O devices (such as a connection failure on a network, or lack of paper in the printer) and in user program (an attempt to access an illegal memory location, or too-great use of CPU time). For each type of error, the OS should take the appropriate action to ensure correct and consistent computing. Debugging facilities can greatly enhance the user's and programmer's abilities to use the system efficiently.
7. **Resource allocation:** when there are multiple users or multiple jobs running at the same time, resources must be allocated to each of them. Many different types of resources are managed by the operating system such as CPU cycles, main memory, file storage and I/O (printers, modems, USB storage drives and other peripheral devices) etc.
8. **Accounting (Logging):** we want to keep track of which users use how much and what kinds of computer resources. These usage statistics may be a valuable tool for researchers who wish to reconfigure the system to improve computing services.
9. **Protection and security:** when several separate processes execute concurrently, it should not be possible for one process to interfere with the

others or with the operating system itself. **Protection involves ensuring that all access to the system resources is controlled.**

**Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts



## Kernel

Kernel is the central module of an OS. It is the part of the operating system that loads first, and it remains in main memory because it stays in memory, it should be as small as possible while still providing all the essential services required by other parts of the OS and applications. The **kernel code is usually loaded into a protected area of memory** to prevent it from being overwritten by programs or other parts of the operating system.

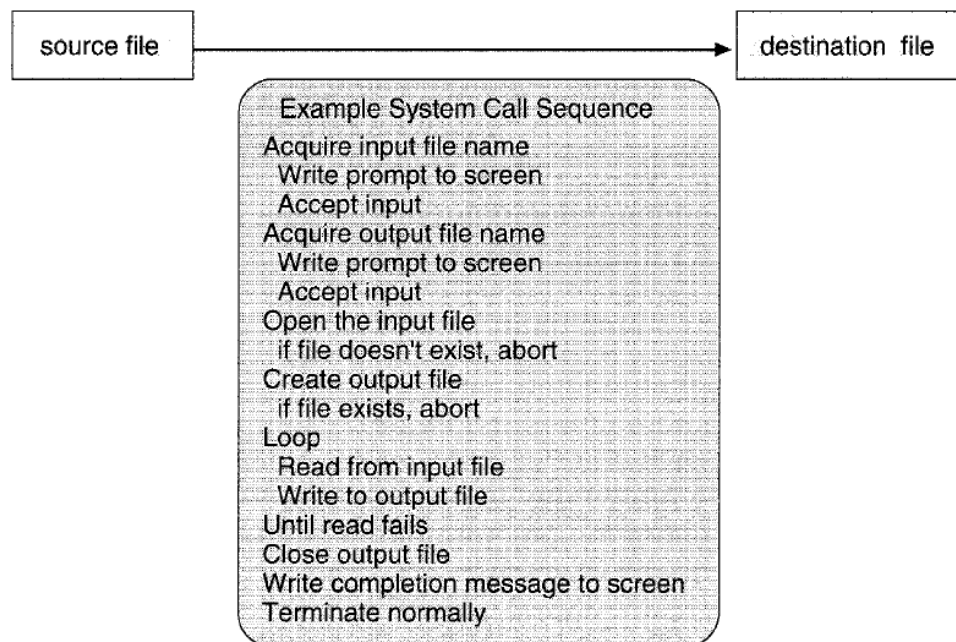
## System Calls

A system call is how a program requests a service from an operating system's kernel. This may include hardware related services (e.g. accessing the hard disk), creating and executing new processes, and communicating with integral kernel services (like scheduling). System calls provide an essential interface between a process and the operating system.

System calls provide an interface to the services made available by an operating

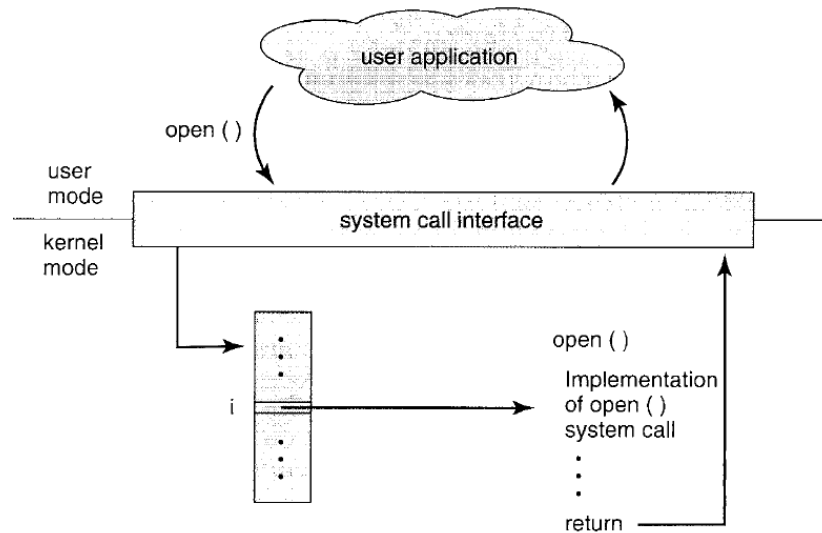
system. These calls are generally available as functions written in C and C++, although certain low-level tasks (for example, tasks where hardware must be accessed directly) may have to be written using assembly-language instructions.

A system call is a mechanism that is used by the application program to request a service from the operating system. They use a **machine-code instruction** that causes the processor to change mode (from user mode to supervisor mode). This is where the operating system performs actions like accessing hardware devices or the memory management unit. Generally, the operating system provides a **library that sits between the operating system and normal programs**.



**Figure 2.1** Example of how system calls are used.





**Figure 2.3** The handling of a user application invoking the `open()` system call.

**Services Provided by System Calls:** system calls can be roughly grouped into 5 categories:

1. Process control
  2. File management
  3. Device management
  4. Information maintenance
  5. Communication
1. **Process control:** A running program needs to be able to halt its execution either normally (end) or abnormally (abort). If a system call is made to terminate the currently running program abnormally, or if the program runs into a problem and causes an error trap, a dump of memory is sometimes taken and an error message generated. The dump is written to disk and may be examined by a debugger—a system program designed to aid the programmer in finding and correcting bugs—to determine the cause of the problem.

- Process control
  - end, abort
  - load, execute
  - create process, terminate process
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory

**File Management:** we need to create and delete files. Either system call requires the name of the file and perhaps some of the file attributes. Once the file is created, we need to open it and use it. We may also read, write, or reposition and finally, we need to close the file, indicating that we are no longer using it. Some operating systems provide many more calls, such as calls for file move and copy. Others might provide an API that performs those operations using code and other system calls, and others might just provide system programs to perform those tasks.

- File management
  - create file, delete file
  - open, close
  - read, write, reposition
  - get file attributes, set file attributes

**Device management:** A process may need several resources to execute such as main memory, disk drives, access to files, and so on. If the resources are available, they can be granted, and control can be returned to the user process. Otherwise, the process will have to wait until sufficient resources are available. **Various resources controlled by the operating system can be thought of as devices.** If there are multiple users of the system, the system may require us to

first request the device, to ensure exclusive use of it. After we are finished with device, we release it.

Once the device has been allocated, we can read, write, and possibly reposition the device, just as we can with files.

- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices

**Information maintenance:** Many system calls exist simply for the purpose of transferring information between the user program and the operating system. For example, most systems have a system call to return the **current time and date**. Other system calls may return information about system, such as **number of current users**, the **version number** of operating system, the **amount of free memory or disk space**, and so on.

- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get process, file, or device attributes
  - set process, file, or device attributes

**Communication:** there are two common mode of inter-process communication: the **message passing model** and **shared-memory model**.

In **message passing model**, the communicating processes exchange messages with one another to transfer information. In here, before communication takes place, a connection must be opened, the name of another communicator must be known (another process on the same computer or a process on another computer connected by a communication link). A host in a network is identified by network

id such as IP address and process are identified by an identifier such as process id. The *get hostid* and *get processid* system calls do this translation. These identifiers are then passed to the general purpose *open* and *close* system calls provided by the *file system* or to specific *open connection* and *close connection* system calls, depending on the system's model of communication. The recipient process must give its permission for communication to take place with an accept *connection call*. The *close connection* system call terminates the communication.

In **shared memory** model, process use *shared memory create* and *shared memory attach* system calls to create and gain access to regions of memory owned by other processes. They can then exchange information by reading and writing data in shared areas.

- Communications
  - create, delete communication connection
  - send, receive messages
  - transfer status information
  - attach or detach remote devices

### **System Programs:**

Another aspect of a modern system is the collection of system programs. Recalling logical computer hierarchy where at the lowest level is *hardware*. Next is the *operating system*, then the *system programs*, and finally the *application programs*. System programs provide a convenient environment for program development and execution. Some of them are simply user interfaces to system calls; others are considerably more complex. They can be divided into these categories:

- **File management.** These programs create, delete, copy, rename, print, dump, list, and generally manipulate files and directories.

- **Status information.** Some programs simply ask the system for the date, time, amount of available memory or disk space, number of users, or similar status information. Others are more complex, providing detailed performance, logging, and debugging information. Typically, these programs format and print the output to the terminal or other output devices or files or display it in a window of the GUI.
- **File modification.** Several text editors may be available to create and modify the content of files stored on disk or other storage devices. There may also be special commands to search contents of files or perform transformations of the text.
- **Programming-language support.** Compilers, assemblers, debuggers and interpreters for common programming languages (such as C, C++, Java, Visual Basic, and PERL) are often provided to the user with the operating system.
- **Program loading and execution.** Once a program is assembled or compiled, it must be loaded into memory to be executed. The system may provide **absolute loaders, relocatable loaders, linkage editors, and overlay loaders**. Debugging systems for either higher-level languages or machine languages are needed as well.

**Note: - [**

**Absolute Loader:** Loads a program **exactly** at the specified memory location.

**Relocatable Loader:** Loads a program **anywhere in memory**, not just a fixed location. The loader **adjusts memory addresses** so the program runs correctly no matter where it is placed.

**Linkage Editor:** Combines multiple program files into one executable file. Resolves **external references** (when one part of the program needs to call another part).

**Overlay Loader:** Helps run **large programs** on computers with **limited memory**. Loads only the necessary parts of the program into memory at a time. When a new part is needed, it replaces the old one (without keeping everything in memory at once).]

- **Communications.** These programs provide the mechanism for creating virtual connections among processes, users, and computer systems. They allow users to send messages to one another's screens, to browse web pages, to send electronic-mail messages, to log in remotely, or to transfer files from one machine to another.

## **System structure**

As OS system is large and complex, a modern operating system must be engineered carefully if it is to function properly and be modified easily. A common approach is to partition the task into small components. Each of these modules should be a well-defined portion of the system, with carefully defined inputs, outputs, and functions. Here we discuss how the components of OS are interconnected and melded into a kernel.

### **1. Monolithic structure:**

The simplest structure for organizing an operating system is no structure at all. That is, place all of the functionality of the kernel into a single, static binary file that runs in a single address space. This approach—known as a **monolithic structure**—is a common technique for designing operating systems. Figure below shows its structure. Everything below the system-call interface and above the physical hardware is the kernel. The kernel provides the file system, CPU scheduling, memory management, and other operating system functions through system calls. Taken in sum, that is an enormous amount of functionality to be combined into one single address space. Early Unix, MS-DOS uses monolithic structure.

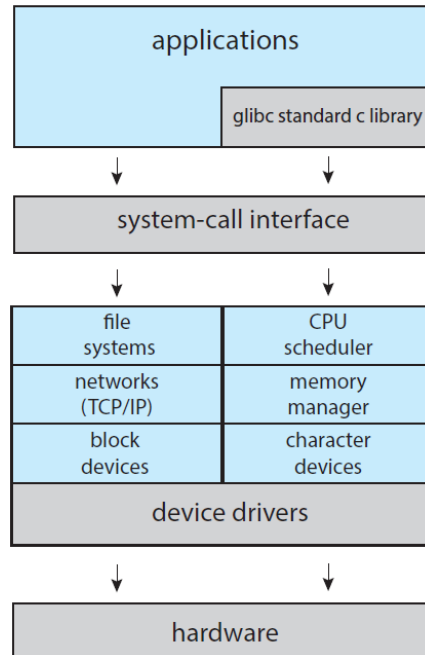
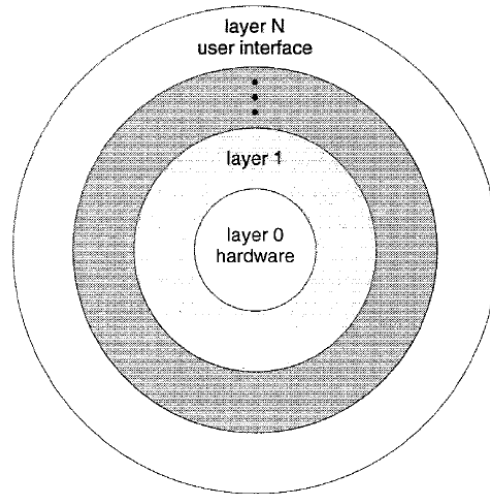


Fig: LINUX System Structure

## 2. Layered approach

With proper hardware support, operating systems can be broken into pieces that are smaller and more appropriate than those allowed by the original MS-DOS or UNIX systems. The operating system can then retain much greater control over the computer and over the applications that make use of that computer.

A system can be made modular in many ways. One method is the layered approach, in which the operating system is broken up into a number of layers (levels). The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface. This layering structure is depicted in Figure below:



The main advantage of the layered approach is simplicity of construction and debugging. The layers are selected so that each uses functions (operations) and services of only lower-level layers. This approach simplifies debugging and system verification. The first layer can be debugged without any concern for the rest of the system, because, by definition, it uses only the basic hardware (which is assumed correct) to implement its functions. Once the first layer is debugged, its correct functioning can be assumed while the second layer is debugged, and so on. If an error is found during the debugging of a particular layer, the error must be on that layer, because the layers below it are already debugged. Thus, the design and implementation of the system is simplified. Each layer is implemented with only those operations provided by lower-level layers. A layer does not need to know how these operations are implemented; it needs to know only what these operations do. Hence, each layer hides the existence of certain data structures, operations, and hardware from higher-level layers.



One reason involves the challenges of appropriately defining the functionality of each layer. In addition, the overall performance of such systems is poor due to the overhead of requiring a user program to traverse through multiple layers to obtain an operating-system service.

3. **Microkernel structure:** This method structures the operating system by removing all non-essential components from the kernel and implementing them as system and user-level programs. The result is a **smaller kernel**. There is little consensus regarding which services should remain in the kernel and which should be implemented in user space. Typically, however, microkernels provide minimal process and memory management, in addition to a communication facility.

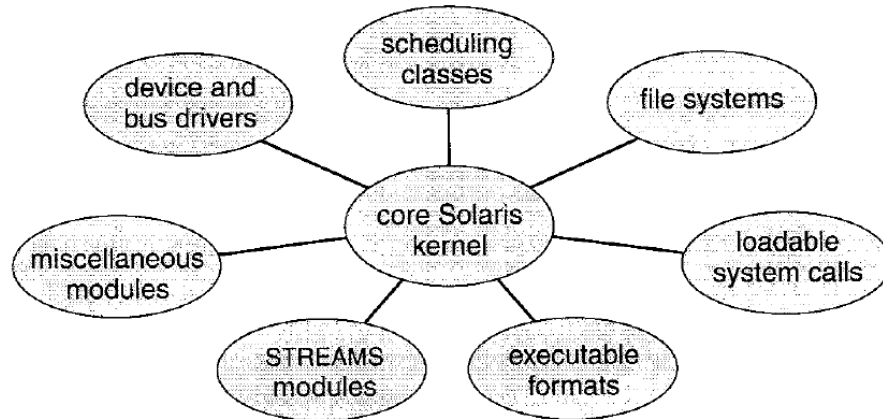
The main function of the microkernel is to provide a communication facility between the client program and the various services that are also running in user space. Communication is provided by message passing. For example, if the client program wishes to access a file, it must interact with the file server. The client program and service never interact directly. Rather, they communicate indirectly by exchanging messages with the microkernel.

One benefit of the microkernel approach is ease of extending the operating system. All new services are added to user space and consequently do not require modification of the kernel. When the kernel does have to be modified, the changes tend to be fewer, because the microkernel is a smaller kernel.

The resulting operating system is easier to port from one hardware design to another. The microkernel also provides more security and reliability, since most services are running as user-rather than kernel-processes. If a service fails, the rest of the operating system remains untouched.

Several contemporary OS are using kernel approach. E.g., Tru64 UNIX, QNX.QNX (Real time OS).

4. **Modules:** Perhaps the best current methodology for operating-system design involves using object-oriented programming techniques to create a modular kernel. Here, the kernel has a set of core components and dynamically links in additional services either during boot time or during run time. Such a strategy uses dynamically loadable modules and is common in modern implementations of UNIX, such as Solaris, Linux, and Mac OS X. For example, the Solaris operating system structure, shown in Figure below, is organized around a core kernel with seven types of loadable kernel modules:



1. Scheduling classes
2. File systems
3. Loadable system calls
4. Executable formats
5. STREAMS modules
6. Miscellaneous
7. Device and bus drivers

Such a design allows the kernel to provide core services yet also allows certain features to be implemented dynamically. For example, device and

bus drivers for specific hardware can be added to the kernel, and support for different file systems can be added as loadable modules. The overall result resembles a layered system in that each kernel section has defined, protected interfaces; but it is more flexible than a layered system in that any module can call any other module. Furthermore, the approach is like the microkernel approach in that the primary module has only core functions and knowledge of how to load and communicate with other modules; but it is more efficient, because modules do not need to invoke message passing in order to communicate.

### **Hybrid System: Self Study**

#### **Virtual machine:**

The fundamental idea behind a virtual machine is to abstract the hardware of a single computer (the CPU, memory, disk drives, network interface cards, and so forth) into several different execution environments, thereby creating the illusion that each separate execution environment is running its own private computer.

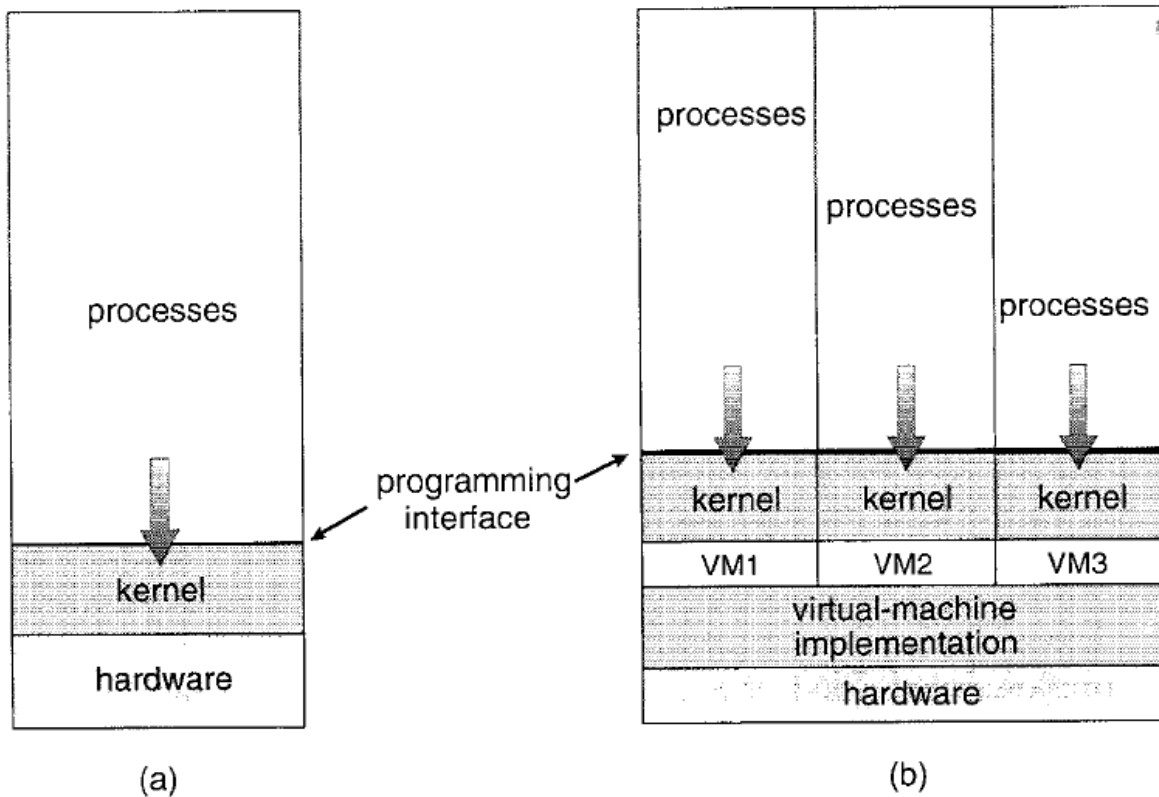
By using **CPU scheduling** and **virtual-memory techniques** an operating system can create the illusion that a process has its own processor with its own (virtual) memory. Normally, a process has additional features, such as system calls and a file system, that are not provided by the bare hardware. The virtual-machine approach does not provide any such additional functionality but rather provides an interface that is identical to the underlying bare hardware. Each process is provided with a (virtual) copy of the underlying computer.

There are several reasons for creating a virtual machine, all of which are fundamentally related to being able to share the same hardware yet run several different execution environments (that is, different operating systems) concurrently.

A major difficulty with the virtual-machine approach involves disk systems. Suppose that the physical machine has three disk drives but wants to support seven virtual machines. Clearly, it cannot allocate a disk drive to each virtual machine, because the virtual-machine software itself will need substantial disk space to provide virtual memory and spooling. The solution is to provide virtual disks-termed **minidisks** in IBM's VM operating system -that are identical in all respects except size. The system implements each minidisk by allocating as many tracks on the physical disks as the minidisk needs. Obviously, the sum of the sizes of all minidisks must be smaller than the size of the physical disk space available.

The virtual-machine concept has several advantages.

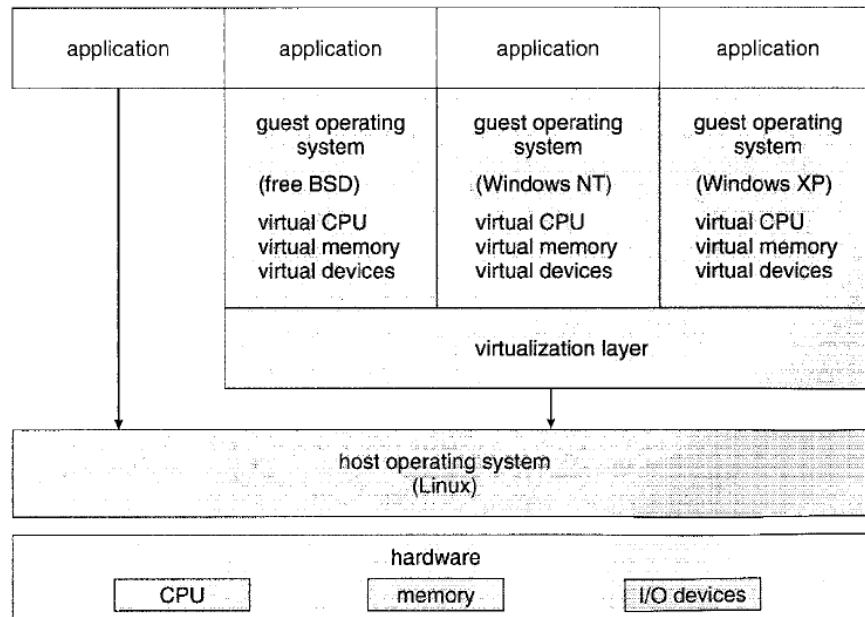
In this environment, there is complete protection of the various system resources. Each virtual machine is completely isolated from all other virtual machines, So, there are no protection problems. At the same time, however, there is no direct sharing of resources. Two approaches to provide sharing have been implemented. First, it is possible to share a minidisk and thus to share files. This scheme is modeled after a physical shared disk but is implemented by software. Second, it is possible to define a network of virtual machines, each of which can send information over the virtual communications network. Again, the network is modeled after physical communication networks but is implemented in software.



**Fig:** System models. (a) Nonvirtual machine. (b) Virtual machine.

## VMware

VMware is a popular commercial application that abstracts Intel 80X86 hardware into isolated virtual machines. VMware runs as an application on a host operating system such as Windows or Linux and allows this host system to concurrently run several different guest operating systems as independent virtual machines.



**Figure 2.16** VMware architecture.